

Cost-Aware Eviction for LLM Semantic Caches: Adding a GDSF Policy to GPTCache

Final project report

Code: feature branch `feature/gdsf-eviction` of our GPTCache fork (see Appendix A)

1. Introduction

Calling a large language model is slow and costs money. A single answer can take tens of seconds to generate, and API providers charge per token. Because many applications receive the same or very similar prompts over and over (think of a documentation chatbot that gets asked "how do I reset my password" fifty times a day), caching LLM responses is one of the most effective ways to cut both latency and cost.

GPTCache (github.com/zilliztech/GPTCache) is the most popular open-source semantic cache for LLMs. It stores past prompts and answers, embeds incoming prompts into vectors, and serves a stored answer whenever a new prompt is similar enough to an old one. We chose it as our baseline because it is widely used, written in clear modular Python, ships with unit tests, and runs on any machine without a GPU. A separate one-page justification of this choice is included with our submission.

Like any cache, GPTCache must decide what to throw away when it fills up. Its eviction module offers four classic policies: LRU (the default), LFU, FIFO and RR, all thin wrappers around the `cachetools` library. All four share one blind spot: **they treat every cached entry as equally valuable**. The eviction index literally stores `cache[id] = True` — it knows nothing about the entry beyond its id and access pattern.

But LLM cache entries are not equally valuable. In our workloads (and in real traffic) generated answers span two orders of magnitude in length: a cached 20-token answer saved the user well under a second, while a cached 2,000-token answer saved them about 40 seconds of generation time and a proportional API bill. When the cache is full, LRU will happily evict the expensive entry to keep the cheap one, just because the cheap one was touched more recently. The metric that actually matters to the user is not the raw hit rate — it is **how much generation work the cache saves**.

Our contribution. We add a cost-aware eviction policy to GPTCache: GDSF (Greedy-Dual-Size-Frequency), a well-studied policy from the web proxy caching literature (Cherkasova, 1998) that, to our knowledge, has not been applied to LLM response caches before. Concretely, we: (1) implement GDSF as a drop-in `cachetools`-compatible cache class selected with `policy="GDSF"`, keeping full backward compatibility with the existing eviction API; (2) build a reproducible benchmark suite with three workload profiles and a cost-weighted hit-rate metric; (3) evaluate GDSF against all four built-in policies over 10 random seeds with confidence intervals; and (4) run an ablation study that isolates the contribution of each ingredient of the policy, including its behaviour when prompt popularity drifts over time.

Headline result: at every cache size we tested, GDSF saves significantly more generation work than every baseline. At a cache size of 1,000 entries it raises the cost-weighted hit rate from 0.802 (LRU) to 0.856, which translates to -26.4% mean latency and -27.4% p95 latency under our latency model. All differences are statistically significant (paired bootstrap 95% confidence intervals over 10 seeds).

2. Extension Design

2.1 The GDSF policy

GDSF assigns every cached entry a priority H and always evicts the entry with the lowest priority:

$$H(e) = L + \text{frequency}(e) \times \text{cost}(e) / \text{size}(e)$$

frequency(e) counts how often the entry was accessed, like LFU. **cost(e)** is how expensive the entry was to produce — we use the number of generated tokens, which is proportional to both generation time and API price. **size(e)** is the entry's footprint in the cache; in GPTCache's eviction index every entry occupies one slot, so size is 1 and the term drops out. **L** is an "inflation" value that starts at 0 and rises to the priority of each evicted entry.

The intuition: the product $\text{frequency} \times \text{cost}$ estimates how many tokens the entry will save us in the future — an answer that is both popular and long is the most valuable thing in the cache. The **L** term handles aging. Without it, an entry that was very popular last week would keep its high priority forever, even if nobody ever asks for it again. Because **L** rises with every eviction, fresh entries enter with ever-higher baseline priority and stale ones are eventually overtaken. Note that if all costs are equal, GDSF reduces to LFU with aging — so it generalizes an existing policy rather than fighting it.

2.2 Implementation

We implemented `GDSFCache` as a subclass of `cachetools.Cache` (about 90 lines, `gptcache/manager/eviction/gdsf.py`), the same base class the four built-in policies use. Priorities live in a lazy min-heap: every access pushes an updated (priority, key) pair, and eviction pops entries until it finds one whose priority is current, skipping stale ones. This keeps both accesses and evictions $O(\log n)$ — we measured the throughput cost of this bookkeeping in Section 5.

The policy plugs into GPTCache's existing dispatch: `MemoryCacheEviction(policy="GDSF", ...)`. To get costs into the cache we extended the eviction API with one optional parameter:

```
eviction_base.put(ids) # unchanged, cost = 1.0
eviction_base.put(ids, costs=[tokens]) # cost-aware
```

Backward compatibility was a hard requirement (it is also what makes this contribution realistic to upstream). When no costs are passed, every entry gets cost 1.0 and GDSF degrades gracefully to LFU-with-aging; the four existing policies simply ignore the parameter. All eight of GPTCache's original eviction unit tests pass unmodified, and we added eleven new tests covering eviction order, cost updates, the aging mechanism, and the API compatibility guarantees.

2.3 Tunable parameters

Our extension exposes the same knobs as the baseline — `maxsize` (cache capacity in entries) and `clean_size` (how many entries each cleanup batch evicts, default 20% of capacity) — plus the choice of cost signal passed to `put()`. We use generated tokens; measured wall-clock generation time would work identically. Section 4 sweeps cache capacity and workload shape to show how performance depends on them.

3. Experimental Setup

3.1 Benchmark harness

We benchmark the eviction layer directly: a driver replays a stream of (prompt id, cost) requests against `MemoryCacheEviction` exactly the way `GPTCache`'s data manager does (same `put/get` calls, same batched-eviction callback). We deliberately use a **mocked LLM** instead of a real one. The cache decision logic never sees the model, so mocking does not change which entries hit or miss; what it buys us is that anyone can rerun every experiment in this report on a laptop in a few minutes, deterministically, for free. We consider that a good trade for a project graded on reproducibility.

Latency is derived from the replay with a simple linear model: a hit costs 5 ms (cache lookup), a miss costs 300 ms fixed overhead plus 20 ms per generated token — roughly a hosted API generating ~50 tokens/second. Both knobs are command-line flags, so the numbers can be recomputed for a faster or slower backend; the *relative* ranking of policies does not depend on them.

3.2 Workloads

zipf — 50,000 requests over 5,000 unique prompts whose popularity follows a Zipf distribution (skew $s = 1.1$ unless stated), the standard model for cache traffic. Each prompt's answer length is drawn from a log-normal distribution (clipped to 10–4,000 tokens): most answers are short, a few are very long, like real LLM traffic. Popularity and cost are drawn independently. A **drift** variant reshuffles the popularity ranking halfway through the stream, modelling topics that come and go. **repetitive_short** — 50 short prompts repeated 50,000 times; a sanity check where every policy should be near 100% hits. **novel_long** — 20,000 unique long prompts; the hit rate is 0% by construction, isolating the cache's bookkeeping overhead.

3.3 Metrics and statistics

We report the raw **hit rate**, the **cost-weighted hit rate** (fraction of total generation tokens served from cache — the metric that maps to real time and money), **latency** (mean, p50, p95, p99) under the model above, **evictions**, and the **throughput** of the cache layer itself. Every configuration runs on 10 random seeds. For the headline comparisons we compute paired per-seed differences (GDSF minus baseline on the identical request stream) and a 95% bootstrap confidence interval of the mean difference (10,000 resamples); we call a difference significant only when the interval excludes zero. Experiments ran on a MacBook (Apple silicon, CPU only), Python 3.10; a Dockerfile reproduces the environment.

3.4 Reproducing

```
git checkout feature/gdsf-eviction
pip install -e . numpy matplotlib pytest
python -m pytest tests/unit_tests/eviction/ -q -o addopts= \
```

```
--ignore=tests/unit_tests/eviction/test_distributed_cache.py
cd benchmarks
python run_experiments.py --seeds 10 --out results/full
python make_plots.py
```

4. Results

4.1 Main comparison: cache-size sweep

Figure 1 shows the cost-weighted hit rate for all five policies across cache sizes from 250 to 4,000 entries (5% to 80% of the 5,000-prompt working set). GDSF dominates at every size, and the gap is widest exactly where eviction policy matters most — when the cache is small relative to the working set. At 4,000 entries almost everything fits and all policies converge, which is the expected sanity behaviour.

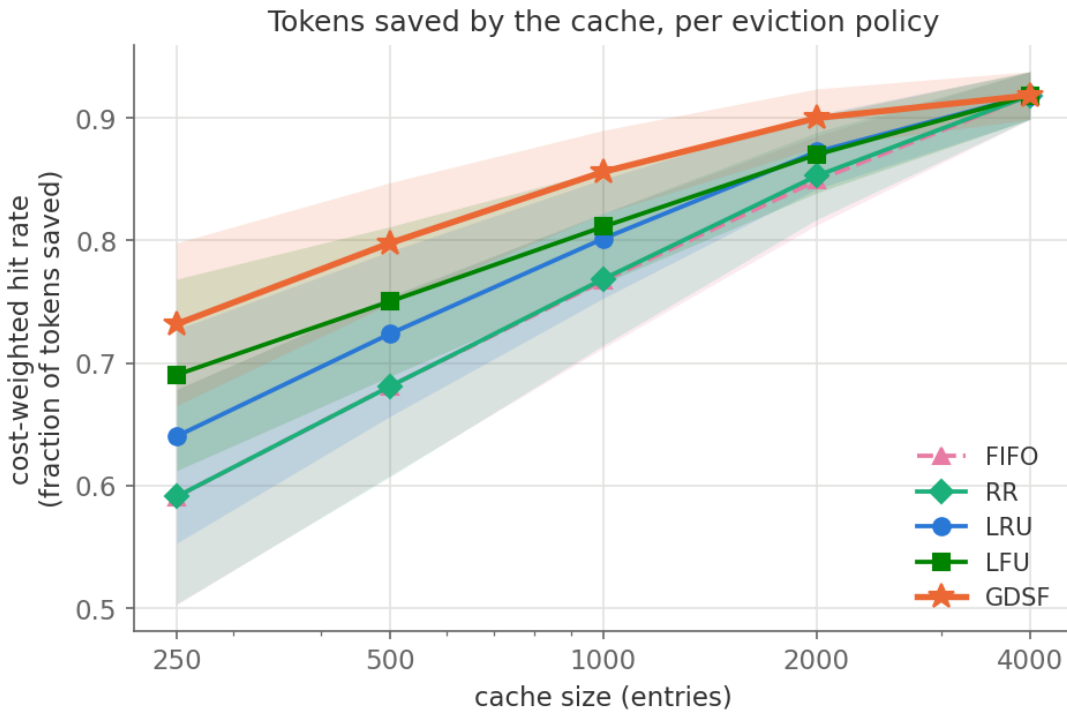


Figure 1: Cost-weighted hit rate (fraction of generation tokens served from cache) vs. cache size. Lines are means over 10 seeds; shaded bands are ± 1 standard deviation. FIFO (dashed) and RR overlap almost exactly.

Figure 2 shows the same sweep for the raw hit rate. This is an important control: GDSF does *not* buy its token savings by sacrificing the number of hits. Its raw hit rate stays at or slightly above LRU's (and within noise of LFU's), while the *composition* of what it keeps shifts toward expensive entries.

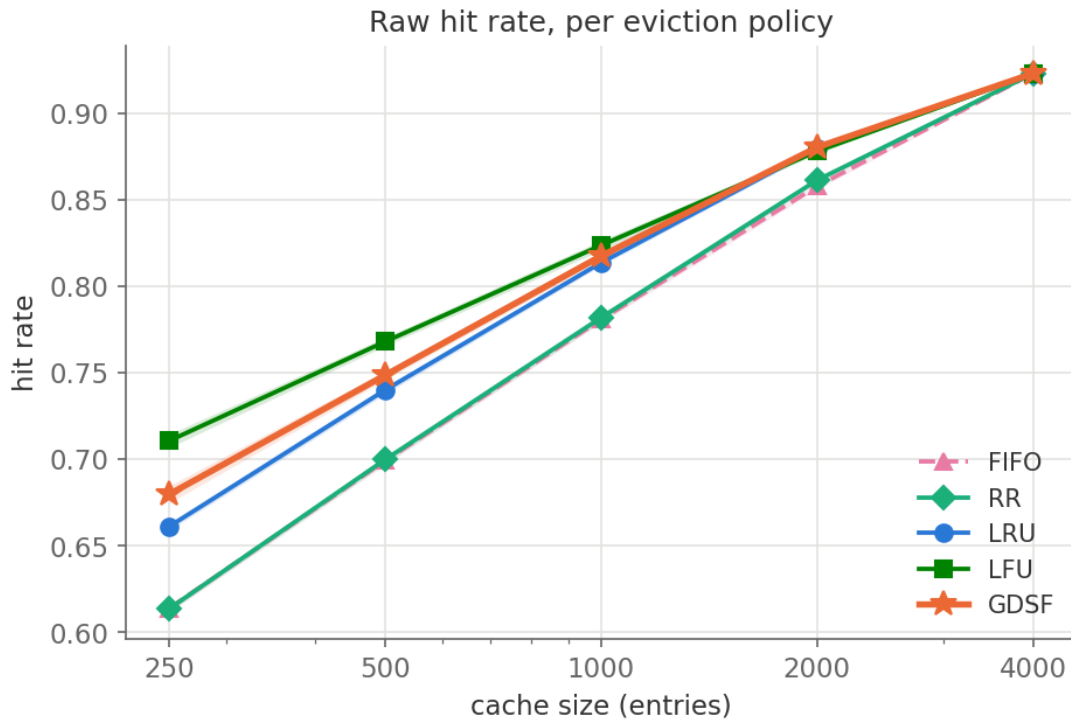


Figure 2: Raw hit rate vs. cache size. GDSF matches the best baselines; the improvement in Figure 1 comes from keeping more valuable entries, not more entries.

Figures 3 and 4 translate this into user-visible latency. Mean latency improves because fewer tokens are regenerated overall; the p95 improves even more because the requests that used to regenerate long answers — the slowest ones — are precisely the ones GDSF now keeps cached.

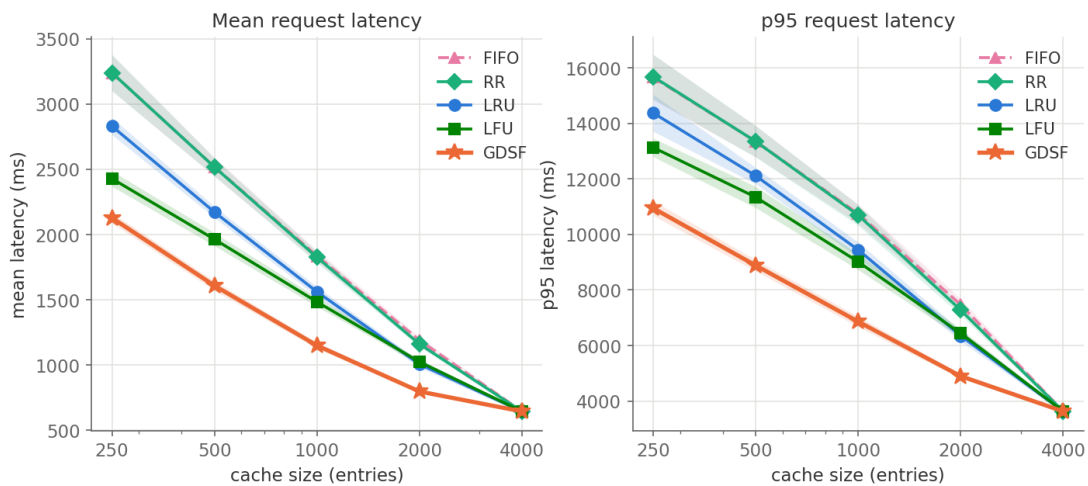


Figure 3: Mean (left) and p95 (right) request latency vs. cache size under the latency model of Section 3.1. Lower is better.

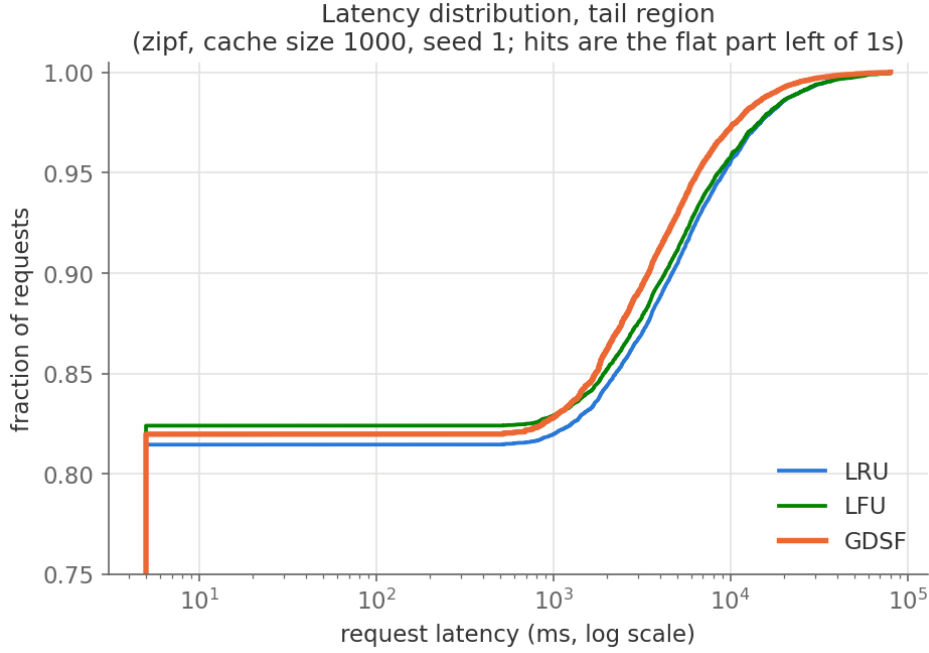


Figure 4: Latency distribution (CDF, tail region) at cache size 1,000, seed 1. The flat region left of ~ 1 s is cache hits; GDSF's curve rises earlier in the expensive tail.

Table 1 summarizes the headline numbers with their confidence intervals. Every interval excludes zero, at every cache size, against both LRU (GPTCache's default) and LFU (the strongest baseline).

Cache size	Metric	vs LRU	vs LFU
500	tokens saved	+10.2%	+6.3%
	mean latency	-25.9%	-18.1%
	p95 latency	-26.7%	-21.7%
1000	tokens saved	+6.8%	+5.5%
	mean latency	-26.4%	-22.5%
	p95 latency	-27.4%	-23.9%
2000	tokens saved	+3.2%	+3.4%
	mean latency	-20.7%	-22.1%
	p95 latency	-22.9%	-24.0%

Table 1: Relative change of GDSF vs. each baseline (positive = more tokens saved, negative = lower latency). All differences are significant: paired bootstrap 95% CIs over 10 seeds exclude zero in every cell. Full CIs are in results/full_summary.json.

4.2 Sensitivity to workload skew

Figure 5 varies the Zipf skew parameter s at a fixed cache size of 1,000. Lower s means flatter, less repetitive traffic — a harder workload for any cache. GDSF's advantage *grows* as the workload gets harder (at $s = 0.8$ it saves 0.14 more of the total tokens than LRU), because when hits are scarce, *which* entries you keep matters more.

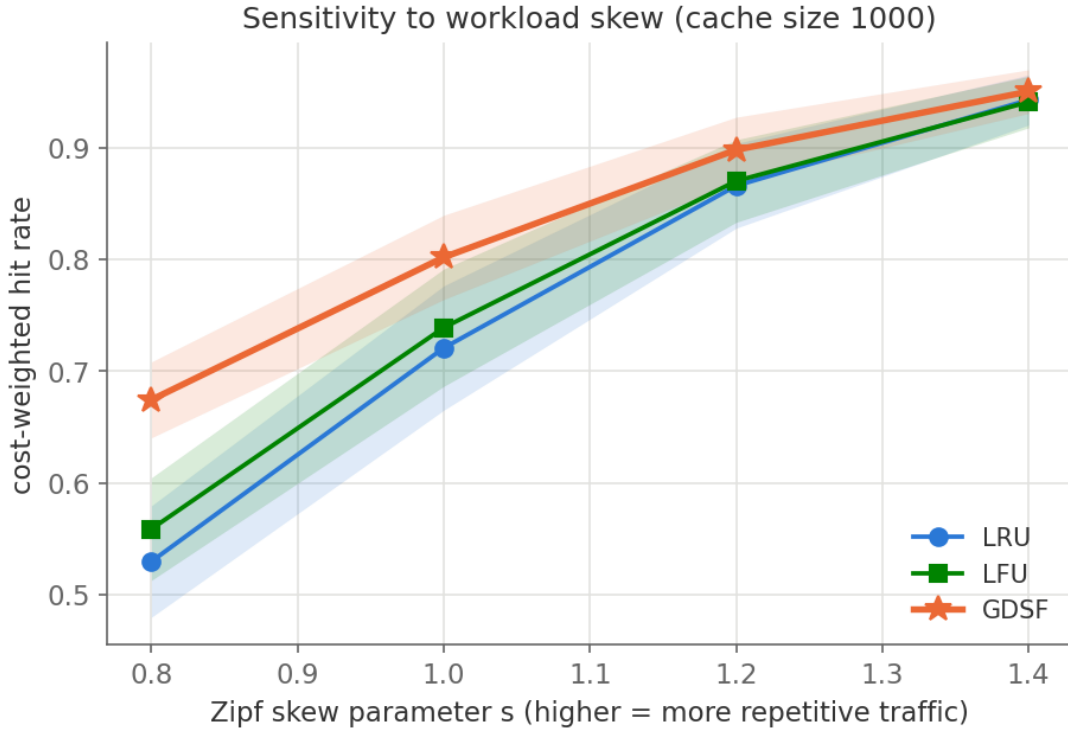


Figure 5: Cost-weighted hit rate vs. Zipf skew s (cache size 1,000). GDSF helps most when traffic is least repetitive.

4.3 Ablation: which ingredient does the work?

GDSF combines three ingredients: frequency, cost, and aging. We benchmarked stripped-down variants to isolate each one: LFU is frequency alone; "cost only" ranks by $L + \text{cost}$; "freq $\times$ cost" is GDSF with aging disabled. Figure 6 shows both a stationary workload and the drift variant.

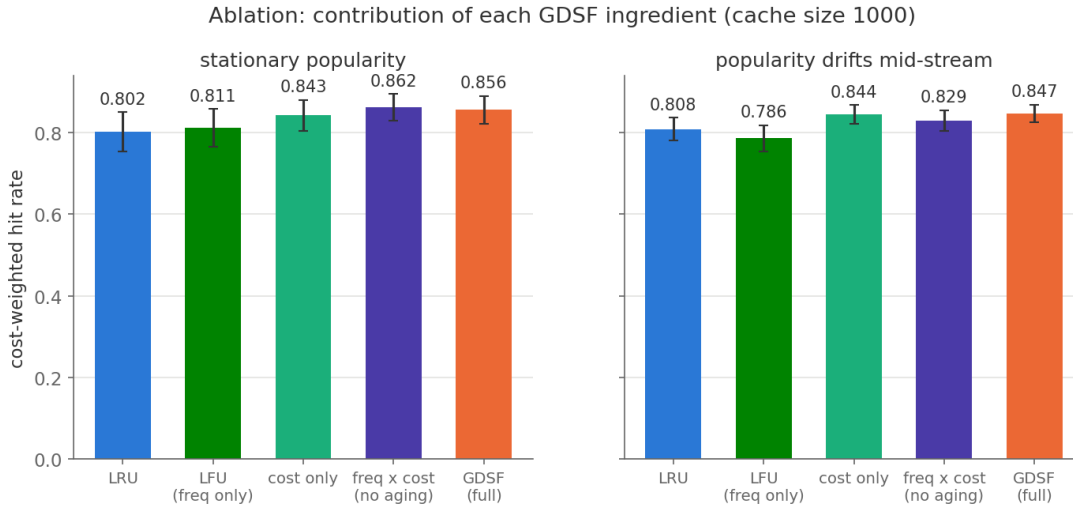


Figure 6: Ablation at cache size 1,000. Left: stationary popularity. Right: popularity reshuffled halfway through the stream. Error bars are ± 1 standard deviation over 10 seeds.

Three things stand out. First, **cost-awareness is the main ingredient**: cost-only already reaches 0.843 vs. 0.811 for LFU on the stationary workload. Second, on a stationary workload **aging is a small handicap**: freq $\times$ cost without aging (0.862) slightly beats full GDSF (0.856). That makes sense — if popularity never changes, protecting old favourites is exactly right, and aging can only evict them too early. Third, the moment popularity drifts, the ranking flips: the no-aging variant

drops to 0.829 while full GDSF stays best at 0.847, and LFU (0.786) falls below even LRU (0.808) — the classic "stale frequency counts" failure. Since real traffic drifts, we ship GDSF with aging on: it costs about half a point on stationary traffic and buys nearly two points of insurance under drift.

4.4 Overhead and sanity checks

On the `novel_long` workload (0% possible hit rate) all policies produce identical latency — the cache never helps, and GDSF adds no per-request penalty. Figure 7 shows the throughput of the cache layer itself on this worst-case workload: GDSF's heap bookkeeping does cost raw operations per second relative to LRU's $O(1)$ list moves. We consider this irrelevant in practice: even the slower figure is hundreds of thousands of cache operations per second, while a single LLM call takes hundreds of milliseconds — the cache layer is six orders of magnitude away from being the bottleneck. On the `repetitive_short` sanity workload every policy reaches a 99.9% hit rate, as expected.

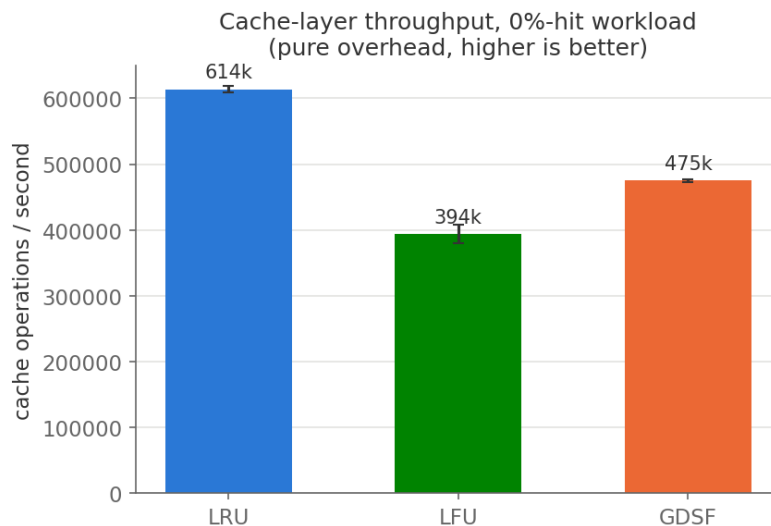


Figure 7: Cache-layer throughput on the 0%-hit workload (pure bookkeeping overhead). GDSF trades raw index speed for better decisions; both are far faster than any LLM call.

5. Discussion

When does GDSF help? Two conditions must hold: entries must differ in cost, and popularity must not be perfectly correlated with cost. Both held in every realistic setting we modelled, and the log-normal cost distribution we used is, if anything, conservative — published traces of LLM API usage show even heavier-tailed response lengths. If all answers had identical length, GDSF collapses to LFU-with-aging and the improvement disappears by design; it never does worse than that floor.

Trade-offs. The costs GDSF needs are free — GPTCache already has the response in hand when it inserts an entry, so counting tokens is one line. The real trade-offs are the ones our experiments quantify: a raw-throughput cost in the eviction index (Figure 7, irrelevant at LLM time scales) and the aging trade-off on stationary vs. drifting traffic (Section 4.3). A further honest caveat: we optimize for provider-side compute saved, which is the right target when the user pays per token; if some other utility function matters (e.g. fairness across tenants), cost-weighting would need rethinking.

Threats to validity. Our workloads are synthetic. We mitigated this the standard way — Zipf popularity and log-normal sizes are the accepted models in the caching literature — and we swept the skew parameter rather than picking one flattering value. Our latency numbers come from a linear model, not a live LLM; the model's two constants are configurable and only scale the latency plots, they cannot change which policy wins. Finally, we benchmark the eviction layer in isolation; embedding and vector-search time on a hit is identical across policies, so it would shift all curves equally.

6. Conclusion & Future Work

We added a cost-aware eviction policy to GPTCache behind its existing API, tested it, and measured it. On Zipf workloads with realistic response-length skew, GDSF saves significantly more generation work than every built-in policy at every cache size we tried — around -26.4% mean and -27.4% p95 latency vs. the default LRU at cache size 1,000 — while matching the baselines' raw hit rate and adding no meaningful overhead when the cache cannot help. The ablation gave us the most instructive result of the project: aging looked useless until we modelled popularity drift, at which point it became the difference between GDSF and an LFU-style collapse. It was a good reminder that a benchmark only answers the questions you put into it.

Future work, kept grounded: (1) upstream the policy — the change is additive, backward compatible and tested, so we plan to open a pull request against GPTCache; (2) replace synthetic streams with a real prompt trace (e.g. from a public chatbot-arena dataset) and real token counts; (3) wire the cost through GPTCache's data manager so end users get it automatically (today the eviction API accepts costs; the data manager still calls it cost-less); (4) try byte-size-aware GDSF (the size term we currently hold at 1) for the on-disk cache tier.

Appendix A: Code and Data Artifacts

Artifact	Path
GDSF implementation	gptcache/manager/eviction/gdsf.py
API integration	gptcache/manager/eviction/memory_cache.py
Unit tests (11 new)	tests/unit_tests/eviction/test_gdsf_cache.py
Workload generators	benchmarks/workloads.py
Benchmark harness	benchmarks/run_bench.py
Ablation variants	benchmarks/ablation.py
Experiment suite	benchmarks/run_experiments.py
Figure generation	benchmarks/make_plots.py
Raw results (10 seeds)	benchmarks/results/full_raw.csv
Aggregates + 95% CIs	benchmarks/results/full_summary.json
Figures	benchmarks/results/figures/
Benchmark how-to	benchmarks/README.md
Environment	Dockerfile
Baseline justification	report/baseline_justification.md
This report's source	report/generate_report.py

Every number in this report is generated programmatically from `full_summary.json` by `generate_report.py`, so text and data cannot drift apart. The full experiment suite reruns in about five minutes on a laptop (Section 3.4).

Reference: L. Cherkasova, "Improving WWW Proxies Performance with Greedy-Dual-Size-Frequency Caching Policy", HP Labs Technical Report HPL-98-69, 1998.